

Facts, Episodes, and Narratives: A Canonical Internal Schema for Phoros

Marcus A. Thomas

Feb 21 2026

1 Overview

This document specifies the internal data schema for Phoros. The schema organizes clinical data into three layers: **Facts** (atomic clinical events), **Episodes** (problem-centric groupings of facts over time), and **Narratives** (authored interpretations of episodes). We refer to this as the FEN schema.

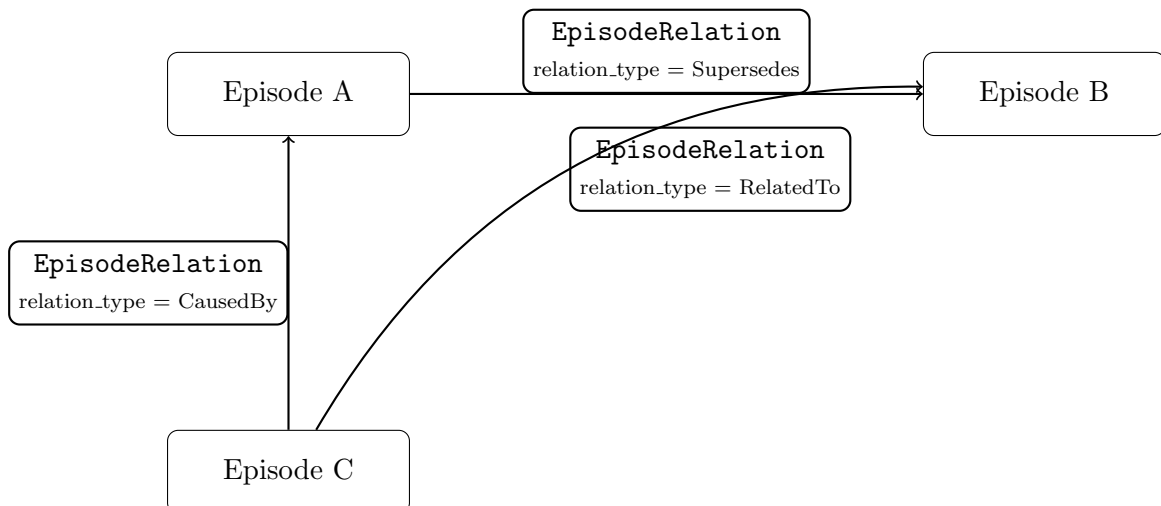
The schema is defined as Rust types. Storage is abstracted: the types do not assume a relational database, document store, or any particular persistence mechanism. The choice of storage backend is a separate decision, downstream of the domain model.

Section 2 defines shared value types. Section 3 introduces traits and explains where polymorphism is and is not warranted. Section 4 explains the key structural decisions and their rationale. Section 5 addresses tradeoffs.

1.1 Entities, Relations, and Graph Representation

The FEN schema adopts a graph-oriented representation. Entities (`Fact`, `ProblemEpisode`, `Narrative`) are modeled as nodes. Relations are modeled as directed, typed edges between nodes. Each edge corresponds to a relation instance such as `EpisodeRelation` or `NarrativeRelation`, with a `relation_type` attribute. Supersession is one such relation type and is represented within this same framework.

This avoids embedding relational structure inside entity definitions. All inter-entity structure is expressed uniformly through authored relations with provenance.



1.2 Fact

A **Fact** is the atomic unit of the schema. It represents a single clinical or scientific or administrative event grounded in evidence: a measurement, a prescription, a procedure, a diagnosis, an imaging result, a note, an encounter, or a billing and insurance event (coverage enrollment, claim submission, adjudication, pre-authorization, appeal). Facts are time-indexed and provenance-rich. They are closer to raw observations and institutional actions than to interpretation.

```
Fact {
  id:                FactId,
  subject_id:        SubjectId,
  occurred_at:        TemporalAnchor,
  code:               Option<CodedValue>,
  payload:            FactPayload,
  status:             FactStatus,
  provenance:         Provenance,
  external_refs:      Vec<ExternalRef>,
}
```

FactPayload is a sum type carrying the type-specific data for each fact variant. This replaces a generic value field with a structured enum whose variants correspond to **FactType**. The rationale for using an enum rather than trait objects for fact-type-specific data is discussed in Section 3.3.

```
FactPayload {
  Measurement { magnitude: f64, unit: String, relevant_note: Option<FactId> },
  Prescription { medication: CodedValue, dosage: Dosage,
    frequency: Frequency, period: Option<TimeInterval>, relevant_note:
Option<FactId> }
  Administration { medication: CodedValue, dosage: Dosage,
    relevant_note: Option<FactId> },
  Procedure { site: Option<CodedValue>,
    laterality: Option<Laterality>, relevant_note: Option<FactId> },
  Diagnosis { relevant_note: Option<FactId> },
  ImagingResult { modality: String, body_site: Option<CodedValue>,
    relevant_note: Option<FactId> },
  LabResult { value: f64, unit: String,
    reference_range: Option<ReferenceRange>, relevant_note: Option<FactId>
},
  Note { content: String, note_type: NoteType },
  Encounter { encounter_type: EncounterType,
    facility: Option<String>, relevant_note: Option<FactId> },
  Referral { to_specialty: String, reason: String },
  Document { document_type: String, content_ref: DocumentRef },
  Coverage { payer: String, plan: String,
    member_id: String, group_id: Option<String>,
    coverage_type: CoverageType, relevant_note: Option<FactId> },
  Claim { claim_id: String, billed_codes: Vec<CodedValue>,
    billed_amount: Money, payer: String, relevant_note: Option<FactId> },
  Adjudication { claim_id: String, allowed_amount: Money,
    patient_responsibility: Money,
    status: AdjudicationStatus, relevant_note: Option<FactId> },
  Payment { claim_id: String, amount: Money,
    paid_by: PayerType, relevant_note: Option<FactId> },
}
```

```

PreAuthRequest { requested_service: CodedValue,
                  payer: String,
                  authorization_id: Option<String>,
                  relevant_note: Option<FactId> },
PreAuthDecision { authorization_id: String,
                  status: AuthorizationStatus,
                  reason: Option<String>,
                  approved_service: Option<CodedValue>,
                  valid_through: Option<Date>, relevant_note: Option<FactId> },
Appeal { original_decision_id: FactId,
         grounds: String,
         supporting_facts: Vec<FactId>, relevant_note: Option<FactId> },
}

```

FactStatus tracks the correctness state of a fact:

```

pub enum FactStatus {
  Active,
  Superseded {
    superseded_by: Author,
    superseded_at: TemporalAnchor,
    replaced_by: Option<FactId>,
    reason: SupersessionReason,
  },
  EnteredInError {
    corrected_by: Author,
    corrected_at: TemporalAnchor,
    replaced_by: Option<FactId>,
  },
}

```

A fact either happened or it did not. Facts are not versioned. If a fact is updated or wrong, its **status** is set to **Superseded** or **EnteredInError**, and a new fact may be created to replace it. The **SupersessionReason** signifies a newer version of the fact exists without implying the original was wrong:

```

pub enum SupersessionReason {
  AiEnrichment,
  ClinicalRefinement, // e.g. a clinician updates a working diagnosis
                     // to confirmed -- not an error, just evolution
}

```

TemporalAnchor represents either a point in time or a period:

```

TemporalAnchor {
  Point(Timestamp),
  Period { start: Timestamp, end: Timestamp },
}

```

1.3 ProblemEpisode

A `ProblemEpisode` is a persistent object representing a coherent clinical problem, question, or management thread over time. An episode may span multiple encounters, involve multiple care sites, and include diverse fact types. It is not a time window. It is the assertion that a set of clinical events belong to the same problem.

```
ProblemEpisode {
  id:          ProblemEpisodeId,
  subject_id:   SubjectId,
  label:        String,
  problem_code: Option<CodedValue>,
  status:       EpisodeStatus,
  onset:        Option<ApproximateDate>,
  authored_by:  Author,
  authored_at:  Timestamp,
  notes:        Option<String>,
}
```

`EpisodeStatus` captures the current clinical phase of the problem:

```
pub struct ResolutionInfo {
  pub at: Option<ApproximateDate>,
}

pub enum EpisodeStatus {
  Active,
  Dormant,
  Resolved(ResolutionInfo),
}
```

1.4 EpisodeMembership

`EpisodeMembership` is the struct that links a fact to an episode. It is a standalone entity, not a list on `ProblemEpisode`. This is a central design decision explained in Section 4.1.

```
MembershipStatus {
  Active,
  Retracted {
    retracted_by: Author,
    retracted_at: TemporalAnchor},
}
```

```
EpisodeMembership {
  id:          MembershipId,
  fact_id:      FactId,
  episode_id:   ProblemEpisodeId,
  role:         FactRole,

  asserted_by:  Author,
  asserted_at:  TemporalAnchor,
```

```
    status:      MembershipStatus,
}
```

FactRole describes the function a fact plays within an episode:

```
FactRole {
    TriggeringSymptom,
    DiagnosticTest,
    Treatment,
    OutcomeMeasure,
    Monitoring,
    Complication,
    Referral,
    Administrative,
    InsuranceAction,
    Other,
}
```

1.5 EpisodeRelation

EpisodeRelation represents an asserted relationship between two episodes. Problems cause other problems, complicate each other, split, and merge. These relationships are not inferred by the system; they are authored claims with provenance.

```
EpisodeRelation {
    id:      RelationId,
    source_episode_id: ProblemEpisodeId,
    target_episode_id: ProblemEpisodeId,
    relation_type: EpisodeRelationType,
    asserted_by: Author,
    asserted_at: Timestamp,
}
```

```
EpisodeRelationType {
    CausedBy,
    Complicates,
    Supersedes,
    SplitFrom,
    MergedInto,
    RelatedTo,
}
```

1.6 Narrative

A **Narrative** is an authored interpretation of an episode. It is an explanation: what happened, what it means, what was decided and why. Multiple narratives per episode is the normal case. A patient, a primary care physician, and a specialist may each author a narrative for the same episode. These narratives may agree, partially overlap, or contradict each other.

```

Narrative {
  id: NarrativeId,
  episode_id: ProblemEpisodeId,
  author: Author,
  authored_at: Timestamp,
  status: NarrativeStatus,
  summary: String,
  sections: Vec<NarrativeSection>,
  decision_points: Vec<DecisionPoint>,
}

```

```

NarrativeStatus {
  Draft,
  Current,
  Superseded,
}

```

1.7 NarrativeRelation

```

NarrativeSupersessionInfo {
  pub note: Option<String>,
}

```

```

ContestationInfo {
  pub scope: Option<ContestationScope>,
  pub rationale: Option<String>,
  pub effective_at: Option<TemporalAnchor>,
}

```

```

ContestationScope {
  Partial,
  Full,
}

```

```

NarrativeRelationType {
  Supersedes(NarrativeSupersessionInfo),
  Contests(ContestationInfo),
  RelatedTo,
}

```

```

NarrativeRelation {
  id: RelationId,
  source_narrative_id: NarrativeId,
  target_narrative_id: NarrativeId,
  relation_type: NarrativeRelationType,
  asserted_by: Author,
}

```

```

    asserted_at:      TemporalAnchor,
  }

```

1.8 NarrativeSection

NarrativeSection provides optional internal structure within a narrative. Not all narratives require sections; the **summary** field on **Narrative** may suffice for simple episodes. Sections are useful when a narrative needs to distinguish phases of a problem or reference specific facts within a larger explanation.

```

NarrativeSection {
  id:          SectionId,
  narrative_id: NarrativeId,
  section_type: SectionType,
  content:     String,
  referenced_facts: Vec<FactId>,
}

```

```

SectionType {
  Onset,
  Workup,
  Diagnosis,
  Treatment,
  Response,
  Progression,
  Resolution,
  OpenQuestions,
}

```

1.9 DecisionPoint

A **DecisionPoint** is a structured record of a clinical decision: what triggered it, what action was taken, and why. It is nested within **Narrative**, not a standalone top-level entity. The rationale for this nesting is discussed in Section 4.3.

```

DecisionPoint {
  id:          DecisionPointId,
  narrative_id: NarrativeId,
  occurred_at:  Timestamp,
  triggering_facts: Vec<FactId>,
  action:      DecisionAction,
  rationale:   String,
  alternatives: Vec<Alternative>,
  author:     Author,
  authored_at: Timestamp,
}

```

DecisionAction is either a reference to a fact (the prescription that was written, the procedure that was performed) or a free-text description when no corresponding fact exists (e.g., a decision

to defer action):

```
DecisionAction {
    FactReference(FactId),
    Description(String),
}
```

1.10 Alternative

Alternative records an option that was considered but not taken. It is a component of **DecisionPoint**.

```
Alternative {
    description:      String,
    reason_not_taken: String,
}
```

2 Shared Value Types

2.1 Provenance

```
Provenance {
    source_system:      Option<String>,
    source_document:    Option<DocumentId>,
    imported_at:        Timestamp,
    author:             Author,
}
```

2.2 Author

```
Author {
    author_type: AuthorType,
    author_id:   Option<AuthorId>,
    display_name: Option<String>,
}
```

```
AuthorType {
    Patient,
    Clinician,
    System,
    AiAssisted,
}
```

2.3 CodedValue

```
CodedValue {
  system:   CodingSystem,
  code:     String,
  display:  String,
}
```

```
CodingSystem {
  SNOMED,
  ICD10,
  LOINC,
  RxNorm,
  CPT,
  Local,
}
```

2.4 ApproximateDate

```
ApproximateDate {
  date:      Date,
  precision: DatePrecision,
}
```

```
DatePrecision {
  Day,
  Month,
  Year,
  Approximate,
}
```

2.5 ExternalRef

```
ExternalRef {
  system:      ExternalSystem,
  resource_type: Option<String>,
  resource_id:  String,
  uri:          Option<String>,
}
```

```
ExternalSystem {
  FHIR,
  OMOP,
  CCDA,
  Other,
}
```

2.6 Billing and insurance value types

```
Money {  
    amount:  f64,  
    currency: String,  
}
```

```
CoverageType {  
    Commercial,  
    Medicare,  
    Medicaid,  
    Tricare,  
    SelfPay,  
    Other,  
}
```

```
AdjudicationStatus {  
    Paid,  
    Denied,  
    PartiallyPaid,  
    Pending,  
}
```

```
PayerType {  
    PrimaryInsurance,  
    SecondaryInsurance,  
    Patient,  
    Other,  
}
```

```
AuthorizationStatus {  
    Approved,  
    Denied,  
    PartiallyApproved,  
    Pending,  
    Appealed,  
}
```

3 Traits

The schema uses traits selectively. Traits capture shared *behavior* that the system needs to dispatch on polymorphically. They are not used merely because two structs share field names. This section identifies three traits that are warranted by concrete system requirements and one case where a trait-based design would be wrong.

3.1 Temporal

The system must place heterogeneous objects on a single timeline. Facts have `occurred_at`. Decision points have `occurred_at`. Episode status changes and narrative revisions have timestamps. A timeline view that interleaves these objects needs a common interface for temporal position.

```
trait Temporal {  
    fn temporal_anchor(&self) -> TemporalAnchor;  
}
```

With this trait, the system can collect `Vec<&dyn Temporal>` from heterogeneous entity types and sort or render them on a unified timeline without matching on every concrete type. `Fact`, `DecisionPoint`, `EpisodeMembership` (via `asserted_at`), and `Narrative` (via `authored_at`) all implement this trait.

The alternative—matching on an enum of all possible timeline entries—works when the set of temporal entities is small and closed. But as the schema grows, the enum becomes a maintenance burden. The trait keeps timeline logic decoupled from the entity definitions.

3.2 Authored

Multiple entities carry authorship: `Fact` (via `Provenance`), `EpisodeMembership`, `EpisodeRelation`, `Narrative`, and `DecisionPoint`. If the system needs to answer “show me everything authored or asserted by Dr. X,” a trait provides a uniform interface across entity types.

```
trait Authored {  
    fn author(&self) -> &Author;  
    fn authored_at(&self) -> Timestamp;  
}
```

This trait is warranted only if cross-entity provenance queries are a real use case. If the system never collects heterogeneous entities into a single provenance-filtered view, the trait adds ceremony without purpose. The current expectation is that provenance queries *are* a core requirement: audit logs, author-filtered views, and consent scoping all require answering “who asserted what, and when” across entity boundaries.

3.3 Where traits are not the right tool

Fact types. The `FactType` enum is a closed set controlled by Phoros. One might be tempted to model `Fact` as a trait with separate structs for `Measurement`, `Prescription`, `Procedure`, and so on, each carrying type-specific fields. This is the wrong approach in Rust.

A closed set of variants is an enum, not a trait hierarchy. Enums provide exhaustive pattern matching: the compiler enforces that every handler covers every variant. They require no heap allocation. They allow value semantics. Trait objects lose all three properties.

The correct design is `FactPayload`, defined in Section ??: a sum type whose variants carry the type-specific data for each fact kind. `Fact` holds the common fields (`id`, `subject_id`, `occurred_at`, `provenance`, etc.) and a `FactPayload` for the variant data. Adding a new fact type means adding a variant to the enum. The compiler will then identify every `match` expression that needs to handle the new variant.

If `FactType` were an open set—if external systems could register new fact types at runtime—trait objects would be justified. That is not the current design.

Shared fields without shared behavior. `EpisodeMembership` and `EpisodeRelation` both have `asserted_by: Author` and `asserted_at: Timestamp`. This structural overlap does not, by itself, justify a trait. A trait is warranted only when the system dispatches generically over the shared interface. If these two types are never collected into the same container and never passed to the same generic function, repeating the fields is clearer than introducing an abstraction with no consumer.

The **Authored** trait covers this case only if cross-entity provenance queries materialize as a requirement. If they do not, the shared fields remain coincidental, and the trait should not exist.

3.4 Summary of trait usage

Pattern	Mechanism	Reason
Timeline placement	Temporal trait	Heterogeneous collection, single sort
Provenance queries	Authored trait	Generic “who authored this” across types
Fact-type-specific data	FactPayload enum	Closed set, exhaustive matching
Shared fields only	Repeat the fields	No behavioral need for abstraction

The guiding principle is: use traits when the system needs to *do the same thing* to different types. Use enums when the system needs to *distinguish* between a fixed set of variants. Repeat fields when structural similarity does not correspond to behavioral polymorphism.

4 Design Decisions

4.1 Why `EpisodeMembership` is a standalone struct

A `ProblemEpisode` does not contain a `Vec<FactId>`. Instead, `EpisodeMembership` is a standalone struct that references both a `FactId` and a `ProblemEpisodeId`. The episode is defined by the set of memberships that point to it, not by an internal collection it owns.

This design has four advantages.

Granular contestability. Assigning a fact to an episode is itself a claim, and claims need provenance. A patient may assert that a lab result belongs to their chronic kidney disease episode. A nephrologist may assert the same lab result belongs to an acute injury episode. Both memberships exist simultaneously with different authors. Neither the episode nor the fact needs to be duplicated. The disagreement lives where it should: in the membership records.

Multi-episode membership. A single fact can belong to multiple episodes without ambiguity. A potassium level may be relevant to both a cardiac arrhythmia episode and a CKD episode. Two memberships, two roles, possibly two different authors. A `Vec<FactId>` on the episode can express multiplicity, but it cannot express *why* or *who said so* for each assignment.

Audit and revision. The question “when was this fact added to this episode, and by whom?” is answered directly by the membership record. If an episode is reorganized—a fact is reassigned from one episode to another—the old membership is retained or marked superseded, and a new one is created. The history of the grouping is preserved.

Role annotation. The `role` field on membership records the function a fact plays within an episode. The same fact may serve as a diagnostic test in one episode and as a monitoring measure in another. This structure does not exist on a flat list.

4.2 Why EpisodeRelation is explicit

The system does not infer that diabetes caused chronic kidney disease. If that relationship is part of the record, someone asserted it. The `EpisodeRelation` struct records who, when, and what type of relationship is claimed.

This is a constraint. It means the system will not automatically link related problems. The benefit is that every inter-episode link has provenance and can be audited, contested, or removed. The alternative—inferring relationships from co-occurrence, shared codes, or temporal proximity—produces links that no one authored and no one can explain.

4.3 Why DecisionPoint is nested under Narrative

A `DecisionPoint` is not a standalone top-level entity. It is a component of `Narrative`. It inherits the authorship and provenance of its parent narrative.

The rationale: a decision rationale is not a fact. It is an explanation of why certain facts led to certain actions. That is narrative. Promoting it to a standalone entity would add a fourth top-level layer (Facts, Episodes, Decisions, Narratives) and raise the question of where decisions sit relative to narratives. Since a well-structured narrative already contains decision rationale, the nesting avoids redundancy.

There is one condition under which promotion is warranted. If decision rationale needs to be *independently contestable*—for instance, a patient disputes the stated reason for a medication change, or a second opinion challenges the reasoning behind a surgical decision—then the `DecisionPoint` may need its own identity and versioning separate from the narrative that contains it. The recommendation is to start nested and promote if this requirement materializes in practice.

4.4 Why billing and insurance are Facts

The schema represents billing events (claims, adjudications, payments), insurance coverage, pre-authorization requests and decisions, and appeals as `Fact` variants rather than as a separate billing domain.

The rationale is twofold. First, these are events. A patient enrolled in a plan at a specific time. A claim was submitted on a specific date. A pre-authorization was denied with a stated reason. Each has a timestamp, provenance, and an author. They satisfy the definition of a `Fact`.

Second, billing events are clinically relevant. A denied pre-authorization for an MRI belongs to the same `ProblemEpisode` as the back pain that prompted it. A claim adjudication that reduces a billed amount is part of the patient’s experience of managing that problem. Excluding billing from the `Fact` layer would force the most financially consequential information outside the representational model that supports patient understanding.

Coverage is a `Fact` because enrollment is an action the patient took at a specific time. The coverage period is a consequence of that action, captured by `TemporalAnchor::Period`. This is analogous to a prescription: an action at time t that produces an effect over a period $[t_1, t_2]$.

Not all billing Facts require episode assignment. A coverage `Fact` is not “about” any particular problem. It exists in the record without `EpisodeMembership` records pointing to it. The schema permits this: `EpisodeMembership` is a separate struct, so unassigned Facts are simply Facts with no

membership records. Claims, pre-authorizations, and appeals, by contrast, are typically episode-specific and should be assigned via `EpisodeMembership` with role `InsuranceAction`.

5 Tradeoffs

5.1 Indirection cost of `EpisodeMembership`

Resolving “all facts in this episode” requires traversing `EpisodeMembership` records rather than reading a `Vec<FactId>` from the episode struct. This is an indirection cost. The magnitude of the cost depends on the storage backend, which the schema does not prescribe.

For read-heavy product surfaces, a denormalized view—episode with its resolved facts—can be materialized and cached. The source of truth remains the membership records, because that is where provenance lives. The denormalized view is a performance optimization, not the data model.

6 Relationship to FHIR and existing standards

FHIR provides `EpisodeOfCare` and `Condition` resources that partially overlap with the episode concept described here. However, `EpisodeOfCare` is scoped to a single managing organization and care period; it is an administrative rather than clinical grouping. `Condition` represents a clinical assertion but does not structurally contain the heterogeneous facts (labs, medications, procedures, notes) associated with managing that condition over time. ICD/SNOMED problem lists enumerate active conditions but do not represent temporal evolution, phase transitions, or interpretive disagreement. The `ProblemEpisode` entity proposed here fills a specific gap: a persistent, cross-encounter, multi-fact grouping that is authorable, and contestable. It is not a replacement for these standards but a layer that organizes what they provide.

The schema maintains interoperability through two mechanisms. First, `ExternalRef` on `Fact` preserves references to FHIR resource URIs, OMOP concept IDs, and C-CDA document identifiers. Second, `CodedValue` uses standard terminologies (SNOMED, ICD-10, LOINC, RxNorm, CPT). The FEN schema does not replace external standards. It ingests from them and can export to them. The layering is:

Ingest external standards → normalize as Facts → lift into Episodes → host Narratives as interpretations.

7 Summary

The FEN schema defines nine entity types organized in three layers, with three traits that provide cross-cutting behavior:

Layer	Entities	Ontological status
Facts	<code>Fact</code>	Observations and actions
Episodes	<code>ProblemEpisode</code> , <code>EpisodeMembership</code> , <code>EpisodeRelation</code>	Structured hypotheses
Narratives	<code>Narrative</code> , <code>NarrativeSection</code> , <code>DecisionPoint</code> , <code>Alternative</code>	Authored interpretations
	Trait	Purpose
	<code>Temporal</code>	Unified timeline placement across entity types
	<code>Authored</code>	Cross-entity provenance queries

The central design commitment is that episode structure and narrative interpretation are first-class objects in the data model, not derived artifacts or unstructured annotations. The central structural decision is that fact-to-episode assignment is a standalone, provenance-bearing record rather than a list on the episode. Traits are used where the system needs polymorphic dispatch over shared behavior; enums are used where the system needs exhaustive matching over a closed set of variants. These choices prioritize contestability, auditability, and fidelity to how clinical reasoning works, at the cost of indirection and storage growth. The schema is defined as Rust types. Storage is abstracted.